

date = '2025-12-25T16:52:11+08:00'

draft = true

title = 'geekchallenge-obfusc3_revenge及引申学习初步控制流平坦化 MBA混淆'

文章推荐: [\[原创\]深入浅出 Ollvm 混淆原理及反混淆技术-Android安全-看雪安全社区 | 专业技术交流与
安全研究论坛](#)

- **指令替换**

是一种通过数学等价变换来增加代码复杂度的混淆技术。它的核心思想是将程序中原本简单的指令（如加法、异或），替换为一段功能等效但逻辑极其晦涩的指令序列

- **虚假控制流**

是 OLLVM 通过向代码中注入永远不会执行的“死代码”块和难以预测的条件跳转，来干扰控制流图（CFG）分析的一种混淆技术

- **控制流平坦化**

是一种旨在摧毁程序结构信息的重度混淆技术。它通过引入一个中央分发器，将原函数中原本层级分明、先后有序的基本块（Basic Blocks）全部“拍扁”，使得它们在控制流图（CFG）上看起来像是在同一个层级上。

具体原理请阅读大佬的文章

- D810插件，得有好用，ida安装好之后ida快捷键Ctrl+Shift+D
- GAMBA，得有，使用见文章[\[分享\]Ollvm 指令替换混淆还原神器：GAMBA 使用指南-Android安全-看雪安全社区 | 专业技术交流与安全研究论坛](#)

obfusc3_revenge

使用D810之后的各个重要函数

main

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
    unsigned int s1[8]; // [rsp+40h] [rbp-280h] BYREF
    void *src; // [rsp+60h] [rbp-260h]
    size_t n; // [rsp+68h] [rbp-258h]
    char s[48]; // [rsp+70h] [rbp-250h] BYREF
    unsigned __int8 a2[256]; // [rsp+A0h] [rbp-220h] BYREF
    unsigned __int8 a1[256]; // [rsp+1A0h] [rbp-120h] BYREF
    unsigned __int8 a3[8]; // [rsp+2A0h] [rbp-20h] BYREF
    __int64 v11; // [rsp+2A8h] [rbp-18h]
    int v12; // [rsp+2BCh] [rbp-4h]

    v12 = 0;
    *(_QWORD *)a3 = 0x8877665540302010LL;
    v11 = 0x2301EFDECDABCAB90LL;
    gen(a1, a2, a3);
    memset(s, 0, 0x24uLL);
    printf("Input the flag: ");
    if ( fgets(s, 36, stdin) )
    {
```

```

n = strcspn(s, "\r\n");
s[n] = 0;
src = s;
if ( n >= 3
    && *((unsigned __int8 *)src) == 239
    && *((unsigned __int8 *)src + 1) == 187
    && *((unsigned __int8 *)src + 2) == 191 )
{
    src = (char *)src + 3;
    n -= 3LL;
}
if ( n && n <= 0x20 )
{
    memset(s1, 0, sizeof(s1));
    memcpy(s1, src, n);
    real_en(s1, a3, a1, a2);
    if ( !memcmp(s1, &main::target_enc, 0x20uLL) )
        printf("Congratulations!\n");
    else
        printf("Try again?\n");
    return 0;
}
else
{
    printf("Try again?\n");
    return 0;
}
}
else
{
    printf("Input error.\n");
    return 1;
}
}

```

gen

```

void __fastcall gen(unsigned __int8 *a1, unsigned __int8 *a2, const unsigned
__int8 *a3)
{
    char v3; // si
    int k; // [rsp+8h] [rbp-2Ch]
    unsigned __int8 v5; // [rsp+Fh] [rbp-25h]
    int j; // [rsp+10h] [rbp-24h]
    unsigned __int8 v7; // [rsp+17h] [rbp-1Dh]
    int i; // [rsp+18h] [rbp-1Ch]

    for ( i = 0; i < 256; ++i )
        a1[i] = i;
    v7 = 0;
    for ( j = 0; j < 256; ++j )
    {
        v3 = a1[j] + v7;
        v7 = a3[j % 16] + v3;
        v5 = a1[j];
    }
}

```

```

    a1[j] = a1[v7];
    a1[v7] = v5;
}
for ( k = 0; k < 256; ++k )
    a2[a1[k]] = k;
}

```

real_en

```

void __fastcall real_en(unsigned int *a1, unsigned __int8 *a2, unsigned __int8
*a3, const unsigned __int8 *a4)
{
    unsigned int (*v4)[4]; // rax
    int v5; // eax
    unsigned int v6; // [rsp+14h] [rbp-ACh]
    unsigned int v7; // [rsp+18h] [rbp-A8h]
    unsigned int v8; // [rsp+1Ch] [rbp-A4h]
    unsigned int v9; // [rsp+20h] [rbp-A0h]
    unsigned int v10; // [rsp+24h] [rbp-9Ch]
    int k; // [rsp+28h] [rbp-98h]
    unsigned int v12; // [rsp+2Ch] [rbp-94h]
    unsigned int v13; // [rsp+30h] [rbp-90h]
    unsigned int v14; // [rsp+34h] [rbp-8Ch]
    unsigned int v15; // [rsp+38h] [rbp-88h]
    unsigned int v16; // [rsp+3Ch] [rbp-84h]
    unsigned int v17; // [rsp+40h] [rbp-80h]
    unsigned int v18; // [rsp+44h] [rbp-7Ch]
    unsigned int v19; // [rsp+48h] [rbp-78h]
    int j; // [rsp+4Ch] [rbp-74h]
    __QWORD v21[5]; // [rsp+50h] [rbp-70h]
    int i; // [rsp+7Ch] [rbp-44h]
    const unsigned __int8 *v23; // [rsp+80h] [rbp-40h]
    unsigned __int8 *v24; // [rsp+88h] [rbp-38h]
    unsigned __int8 *v25; // [rsp+90h] [rbp-30h]
    unsigned int *v26; // [rsp+98h] [rbp-28h]

    v26 = a1;
    v25 = a2;
    v24 = a3;
    v23 = a4;
    en1((unsigned __int8 *)a1, a2, 32);
    for ( i = 0; i < 32; ++i )
        *((_BYTE *)v26 + i) = v24[*((unsigned __int8 *)v26 + i)];
    for ( j = 0; j < 32; ++j )
        *((_BYTE *)v21 + j) = *((_BYTE *)v26 + (unsigned __int8)perm[j]);
    v4 = (unsigned int (*)(4))v26;
    *(__QWORD *)v26 = v21[0];
    *(__QWORD *)&(*v4)[2] = v21[1];
    *(__QWORD *)&(*v4)[4] = v21[2];
    *(__QWORD *)&(*v4)[6] = v21[3];
    v19 = *v26;
    v18 = v26[1];
    v17 = v26[2];
    v16 = v26[3];
    v15 = v26[4];

```

```

v14 = v26[5];
v13 = v26[6];
v12 = v26[7];
for ( k = 0; k < 4; ++k )
{
    v10 = *(_DWORD *)&v25[4 * (k % 4)];
    v9 = F(v18, v10, v24) ^ (v19 & 0xA048412B | ~v19 & 0x51030000 | 0x8A430C0) ^
(~v19 & 0x6108E14 | v19 & 0x8A430C0 | 0x51030000) ^ 0xEB4BED4;
    v19 = v18;
    v18 = v9;
    v5 = F(v16, v10, v24);
    v8 = (v17 & 0x805110C0 | ~v17 & 0x2C28E804 | 0x43000638) ^ (~v17 & 0x10860103
| v17 & 0x43000638 | 0x2C28E804) ^ (v5 & 0x2C3060C4 | ~v5 & 0x80499800 |
0x40860723) ^ (~v5 & 0x13000018 | v5 & 0x40860723 | 0x80499800);
    v17 = v16;
    v16 = v8;
    v7 = v15 ^ F(v14, v10, v24);
    v15 = v14;
    v14 = v7;
    v6 = F(v12, v10, v24) ^ v13;
    v13 = v12;
    v12 = v6;
}
*v26 = v19;
v26[1] = v18;
v26[2] = v17;
v26[3] = v16;
v26[4] = v15;
v26[5] = v14;
v26[6] = v13;
v26[7] = v12;
}

```

用GAMBA去除混淆

```

PS E:\GAMBA-main\GAMBA-main\src> python3 simplify.py "(a & 0xA048412B | ~a & 0x51030000 | 0x8A430C0) ^ (~a & 0x6108E14
| a & 0x8A430C0 | 0x51030000) ^ 0xEB4BED4"
E:\GAMBA-main\GAMBA-main\src\simplify.py:890: SyntaxWarning: invalid escape sequence '\+'
l = re.split("(\\+)", expr)
E:\GAMBA-main\GAMBA-main\src\simplify.py:1084: SyntaxWarning: invalid escape sequence '\+'
return not bool(re.search("(\\+)", expr)) and bool(re.search("([&|^])", expr))
*** Expression (a & 0xA048412B | ~a & 0x51030000 | 0x8A430C0) ^ (~a & 0x6108E14 | a & 0x8A430C0 | 0x51030000) ^ 0xEB4BED
4
*** ... simplified to a
PS E:\GAMBA-main\GAMBA-main\src> python3 simplify.py "(v17 & 0x805110C0 | ~v17 & 0x2C28E804 | 0x43000638) ^ (~v17 & 0x10
860103 | v17 & 0x43000638 | 0x2C28E804) ^ (v5 & 0x2C3060C4 | ~v5 & 0x80499800 | 0x40860723) ^ (~v5 & 0x13000018 | v5 & 0
x40860723 | 0x80499800)"
E:\GAMBA-main\GAMBA-main\src\simplify.py:890: SyntaxWarning: invalid escape sequence '\+'
l = re.split("(\\+)", expr)
E:\GAMBA-main\GAMBA-main\src\simplify.py:1084: SyntaxWarning: invalid escape sequence '\+'
return not bool(re.search("(\\+)", expr)) and bool(re.search("([&|^])", expr))
*** Expression (v17 & 0x805110C0 | ~v17 & 0x2C28E804 | 0x43000638) ^ (~v17 & 0x10860103 | v17 & 0x43000638 | 0x2C28E804)
^ (v5 & 0x2C3060C4 | ~v5 & 0x80499800 | 0x40860723) ^ (~v5 & 0x13000018 | v5 & 0x40860723 | 0x80499800)
*** ... simplified to v5^v17

```

再简单合并一下

```

void __fastcall real_en(unsigned int *a1, unsigned __int8 *a2, unsigned __int8
*a3, const unsigned __int8 *a4)
{
    unsigned int (*v4)[4]; // rax
    int v5; // eax
    unsigned int v6; // [rsp+14h] [rbp-ACh]
    unsigned int v7; // [rsp+18h] [rbp-A8h]
    unsigned int v8; // [rsp+1Ch] [rbp-A4h]

```

```

unsigned int v9; // [rsp+20h] [rbp-A0h]
unsigned int v10; // [rsp+24h] [rbp-9Ch]
int k; // [rsp+28h] [rbp-98h]
unsigned int v12; // [rsp+2Ch] [rbp-94h]
unsigned int v13; // [rsp+30h] [rbp-90h]
unsigned int v14; // [rsp+34h] [rbp-8Ch]
unsigned int v15; // [rsp+38h] [rbp-88h]
unsigned int v16; // [rsp+3Ch] [rbp-84h]
unsigned int v17; // [rsp+40h] [rbp-80h]
unsigned int v18; // [rsp+44h] [rbp-7Ch]
unsigned int v19; // [rsp+48h] [rbp-78h]
int j; // [rsp+4Ch] [rbp-74h]
_QWORD v21[5]; // [rsp+50h] [rbp-70h]
int i; // [rsp+7Ch] [rbp-44h]
const unsigned __int8 *v23; // [rsp+80h] [rbp-40h]
unsigned __int8 *v24; // [rsp+88h] [rbp-38h]
unsigned __int8 *v25; // [rsp+90h] [rbp-30h]
unsigned int *v26; // [rsp+98h] [rbp-28h]

v26 = a1;
v25 = a2;
v24 = a3;
v23 = a4;
en1((unsigned __int8 *)a1, a2, 32);
for ( i = 0; i < 32; ++i )
    *((_BYTE *)v26 + i) = v24[*((unsigned __int8 *)v26 + i)];
for ( j = 0; j < 32; ++j )
    *((_BYTE *)v21 + j) = *((_BYTE *)v26 + (unsigned __int8)perm[j]);
v4 = (unsigned int (*)(4))v26;
*(_QWORD *)v26 = v21[0];
*(_QWORD *)&(*v4)[2] = v21[1];
*(_QWORD *)&(*v4)[4] = v21[2];
*(_QWORD *)&(*v4)[6] = v21[3];
v19 = *v26;
v18 = v26[1];
v17 = v26[2];
v16 = v26[3];
v15 = v26[4];
v14 = v26[5];
v13 = v26[6];
v12 = v26[7];
for ( k = 0; k < 4; ++k )
{
    v10 = *(_DWORD *)&v25[4 * (k % 4)];
    v9 = F(v18, v10, v24) ^ v19
    v19 = v18;
    v18 = v9;
    v8 = v17 ^ F(v16, v10, v24);
    v17 = v16;
    v16 = v8;
    v7 = v15 ^ F(v14, v10, v24);
    v15 = v14;
    v14 = v7;
    v6 = F(v12, v10, v24) ^ v13;
    v13 = v12;
    v12 = v6;
}

```

```

}
*v26 = v19;
v26[1] = v18;
v26[2] = v17;
v26[3] = v16;
v26[4] = v15;
v26[5] = v14;
v26[6] = v13;
v26[7] = v12;
}

```

en1

```

__int64 __fastcall en1(unsigned __int8 *a1, const unsigned __int8 *a2, signed int
a3)
{
    __int64 result; // rax
    signed int i; // [rsp+10h] [rbp-30h]

    for ( i = 0; ; ++i )
    {
        result = (unsigned int)i;
        if ( i >= a3 )
            break;
        a1[i] = ~(a2[i % 16] | ~a1[i]) & ~(a2[i % 16] & ~a1[i]);
    }
    return result;
}

```

F和not_en

```

__int64 __fastcall F(int a1, unsigned int a2, const unsigned __int8 *a3)
{
    unsigned int v3; // r13d
    int v4; // r12d
    unsigned int v5; // r15d
    unsigned int v6; // edx
    unsigned int v7; // r10d
    unsigned int v9; // [rsp+4h] [rbp-3Ch]

    HIBYTE(v9) = a3[(unsigned __int8)(a1 ^ a2)];
    BYTE2(v9) = a3[(unsigned __int16)(a1 ^ a2) >> 8];
    BYTE1(v9) = a3[(unsigned __int8)((a1 ^ a2) >> 16)];
    LOBYTE(v9) = a3[(a1 ^ a2) >> 24];
    v3 = ((unsigned __int8)v9 << 24) & 0xE18ACD3F;
    v4 = ~(BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) | 0x1E7532C0;
    v5 = ~(v3 | ~((unsigned __int8)v9 << 24) & 0x1E7532C0) & ~(v4 | (BYTE1(v9) <<
16) | (BYTE2(v9) << 8) | HIBYTE(v9) | 0xE18ACD3F) | (((BYTE1(v9) << 16) |
(BYTE2(v9) << 8) | HIBYTE(v9)) & 0xA102C816 | ~(BYTE1(v9) << 16) | (BYTE2(v9) <<
8) | HIBYTE(v9)) & 0x4080529 | 0x1E353280) ^ ~(BYTE1(v9) << 16) | (BYTE2(v9)
<< 8) | HIBYTE(v9)) & 0x400040 | ((BYTE1(v9) << 16) | (BYTE2(v9) << 8) |
HIBYTE(v9)) & 0x1E353280 | 0x4080529));
    v6 = ((unsigned __int8)v9 << 24) ^ ((BYTE1(v9) << 16) | (BYTE2(v9) << 8) |
HIBYTE(v9)) | ~_byteswap_ulong(v9);

```

```

v7 = (v5 | ~(~(v4 | (BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9) |
0xE18ACD3F) | (((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0xA102C816 |
~((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x40880529 | 0x1E353280) ^
(~((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x400040 | ((BYTE1(v9) <<
16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x1E353280 | 0x40880529)) & (v3 | ~
((unsigned __int8)v9 << 24) & 0x1E7532C0)) & 0x80363BBB;
return not_en(
    ~v6 & (v5 | ~(~(v4 | (BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)
| 0xE18ACD3F) | (((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0xA102C816
| ~((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x40880529 | 0x1E353280)
^ (~((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x400040 | ((BYTE1(v9)
<< 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x1E353280 | 0x40880529)) & (v3 | ~
((unsigned __int8)v9 << 24) & 0x1E7532C0)) | v6 ^ 0x80363BBB ^ (v7 | ~(v5 | ~(~
(v4 | (BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9) | 0xE18ACD3F) |
(((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0xA102C816 | ~((BYTE1(v9)
<< 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x40880529 | 0x1E353280) ^ (~
((BYTE1(v9) << 16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x400040 | ((BYTE1(v9) <<
16) | (BYTE2(v9) << 8) | HIBYTE(v9)) & 0x1E353280 | 0x40880529)) & (v3 | ~
((unsigned __int8)v9 << 24) & 0x1E7532C0)) & 0x7FC9C444),
    5);
}

__int64 __fastcall not_en(unsigned int a1, char a2)
{
    int v2; // esi
    unsigned int v3; // edi

    v2 = a1 << a2;
    v3 = a1 >> (32 - a2);
    return v3 & v2 | (v3 & 0x18B2001 | ~v3 & 0x643081A4 | 0x82444848) ^ ~v3 &
0x18001612 ^ (v3 & 0x82444848 | 0x643081A4) ^ v2 ^ 0x9A445E5A;
}

```

剩下没有去除的混淆可以交给GAMBA也可以直接给ai

ds去混淆之后的结果

```

__int64 __fastcall en1(unsigned __int8 *a1, const unsigned __int8 *a2, signed int
a3) {
    for (int i = 0; i < a3; ++i) {
        a1[i] = a1[i] ^ a2[i % 16];
    }
    return a3;
}

```

```

uint32_t F(int a1, unsigned int a2, const uint8_t *sbox) {
    uint32_t v = a1 ^ a2; // 异或轮密钥

    // 每个字节通过S盒
    uint8_t b0 = sbox[(v >> 0) & 0xFF]; // 原HIBYTE(v9)
    uint8_t b1 = sbox[(v >> 8) & 0xFF]; // 原BYTE2(v9)
    uint8_t b2 = sbox[(v >> 16) & 0xFF]; // 原BYTE1(v9)
    uint8_t b3 = sbox[(v >> 24) & 0xFF]; // 原LOBYTE(v9)

    // 组合成32位值（注意字节顺序）
}

```

```

uint32_t combined = (b0 << 24) | (b1 << 16) | (b2 << 8) | b3;

// 循环左移5位
return (combined << 5) | (combined >> (32 - 5));
}

uint32_t not_en(unsigned int a1, char a2) {
    // 简单的循环左移
    return (a1 << a2) | (a1 >> (32 - a2));
}

```

密文0xCA, 0x5A, 0x96, 0xFF, 0x08, 0x49, 0x72, 0x39, 0x36, 0x18, 0x13, 0x8A, 0x14, 0xC0, 0x0C, 0x78, 0xF8, 0x7C, 0x49, 0xC7, 0xBE, 0xE8, 0x91, 0xED, 0x7F, 0xB0, 0x02, 0xAD, 0x77, 0x74, 0xD4, 0x34

perm 0x1F, 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10, 0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A, 0x1B, 0x1C, 0x1D, 0x1E

写出exp

```

def b2nle(b, n):
    """字节数组转小端序整数列表"""
    return [int.from_bytes(b[i:i+n], byteorder='little', signed=False)
            for i in range(0, len(b), n)]

def n2ble(na, n):
    """小端序整数列表转字节数组"""
    b = bytearray()
    for a in na:
        b.extend(a.to_bytes(n, byteorder='little'))
    return b

def rol(b, shift):
    """32位循环左移"""
    shift &= 31
    return ((b << shift) | (b >> (32 - shift))) & 0xFFFFFFFF

def gen(sbox, inv_sbox, key):
    """生成S盒和逆S盒"""
    for i in range(256):
        sbox[i] = i

    t = 0
    for j in range(256):
        t = (t + key[j % 16] + sbox[j]) & 0xFF
        sbox[j], sbox[t] = sbox[t], sbox[j]

    for k in range(256):
        inv_sbox[sbox[k]] = k

def F(dm, dk, sbox):

```



```

"""Feistel的F函数"""
dm ^= dk
bm = n2ble([dm], 4)
for i in range(4):
    bm[i] = sbbox[bm[i]]
dm = b2nle(bm, 4)[0]
return rol(dm, 5)

def decrypt_flag():
    """解密主函数"""
    # 密钥
    key = bytearray.fromhex("102030405566778890ABBCCDDEEF0123")
    dkey = b2nle(key, 4)

    # 密文
    enc =
bytearray.fromhex("CA5A96FF084972393618138A14C00C78F87C49C7BEE891ED7FB002AD7774D4
34")

    # 生成S盒
    sbbox = bytearray(256)
    inv_sbbox = bytearray(256)
    gen(sbbox, inv_sbbox, key)

    # 1. 将密文转换为小端序的uint32列表并反转
    denc = b2nle(enc, 4)[::-1]

    # 2. 逆Feistel网络（4轮）
    for i in range(3, -1, -1):
        for j in range(0, 8, 2):
            L = denc[j]
            R = denc[j + 1]
            denc[j] = R
            denc[j + 1] = L ^ F(R, dkey[i], sbbox)

    # 3. 反转回来并转换为字节数组
    denc = denc[::-1]
    benc = n2ble(denc, 4)

    # 4. 循环左移1位（逆循环右移）
    rec = benc[1:] + benc[:1]

    # 5. 逆S盒
    for i in range(32):
        rec[i] = inv_sbbox[rec[i]]

    # 6. 异或密钥
    for i in range(32):
        rec[i] ^= key[i % 16]

    return rec

# 执行解密
flag_bytes = decrypt_flag()
flag_clean = flag_bytes.rstrip(b'\x00')

```

```
# 输出结果
print(f"解密结果 (hex): {flag_bytes.hex()}")
print(f"解密结果 (raw): {flag_bytes}")

try:
    flag_str = flag_clean.decode('utf-8')
    print(f"\nFlag: {flag_str}")
except:
    print(f"\nFlag (原始字节): {flag_clean}")
```

SYC{Then_you_are_1mpressivse}

flutter

直接看so，看到了了一组非常像密文的数据

```
.rodata:000000000000FC00 byte_FC00          db 57h, 51h, 47h, 7Fh, 44h, 29h, 75h, 2
dup(76h), 37h
.rodata:000000000000FC00                  ; DATA XREF:
sub_1A210+4↓o
.rodata:000000000000FC0A          db 68h, 5Bh, 67h, 3Ah, 6Ch, 5Bh, 55h,
77h, 37h, 5Bh, 65h
.rodata:000000000000FC15          db 74h, 37h, 68h, 71h, 73h, 42h, 65h,
68h, 37h, 19h, 7Dh
```

交叉引用找到

```
void __fastcall sub_1A210(__int64 a1, __int64 a2)
{
    if ( (*(_BYTE *))(a1 + 24) ^ byte_FC00[*(unsigned __int8 *)](a2 + 1)] != 'R' )
        *(_BYTE *))(a1 + 28) = 1;
}
```

在找到sub_19550

```
char __fastcall sub_19550(__int64 a1)
{
    int v1; // r13d
    unsigned __int8 *v2; // r14
    __int64 v3; // r15
    unsigned __int64 v4; // rbp
    unsigned __int64 v5; // rax
    void (__fastcall *v6)(__int64); // rax
    int v7; // ecx
    int v8; // eax
    char v9; // dl

    v2 = (unsigned __int8 *)ptr;
    if ( *(&ptr + 1) == ptr )
        return 0;
    v3 = 0LL;
    v4 = 0LL;
```

```

do
{
    v5 = v2[v3];
    if ( v5 > 0xF || (v6 = (void (__fastcall *)(__int64))funcs_195B4[v5]) == 0LL
)
    {
        *(_BYTE *)(a1 + 28) = 1;
        v8 = 2;
        v9 = 0;
LABEL_9:
        v7 = v1;
        goto LABEL_10;
    }
    v6(a1);
    v7 = v2[v3];
    v8 = 1;
    if ( v7 == 9 )
    {
        v9 = 0;
        v7 = 0;
    }
    else
    {
        if ( v7 == 8 )
        {
            LOBYTE(v7) = *(_BYTE *)(a1 + 28) == 0;
            v9 = 0;
            goto LABEL_10;
        }
        v9 = 0;
        v7 = 0;
        if ( !*(_BYTE *)(a1 + 28) )
        {
            v9 = 1;
            v8 = 0;
            goto LABEL_9;
        }
    }
}
LABEL_10:
    if ( !v9 )
        return v7 & (v8 != 2);
    ++v4;
    v2 = (unsigned __int8 *)ptr;
    v3 += 3LL;
    v1 = v7;
}
while ( v4 < 0xAAAAAAAAAAAAAAAABLL * ((__BYTE *)(&ptr + 1) - (__BYTE *)ptr) );
v8 = 2;
return v7 & (v8 != 2);
}

```

```

.data.rel.ro:0000000000039E78 func_195B4      dq offset sub_1A1B0      ; DATA
XREF: sub_19550+4B↑o
.data.rel.ro:0000000000039E78                                     ;
sub_19550+52↑r
.data.rel.ro:0000000000039E80      dq offset sub_1A1D0
.data.rel.ro:0000000000039E88      dq offset sub_1A1F0
.data.rel.ro:0000000000039E90      dq offset sub_1A200
.data.rel.ro:0000000000039E98      dq offset sub_1A210
.data.rel.ro:0000000000039EA0      dq offset sub_1A230
.data.rel.ro:0000000000039EA8      dq offset sub_1A240
.data.rel.ro:0000000000039EB0      dq offset nullsub_2
.data.rel.ro:0000000000039EB8      dq offset sub_1A260
.data.rel.ro:0000000000039EC0      dq offset sub_1A270

```

可以用vm的方法分析，也可以直接爆破

```

# 密文（与你的原始密文一致，16进制字符串转bytearray）
E =
bytearray.fromhex("5751477F442975767637685B673A6C5B5577375B6574376871734265683719
7D")
known_prefix = b"SYC{"

# 第一部分: ((c + i) ^ j) & 0xFF
print("=== 爆破: 先加后异或 ((c + i) ^ j) ===")
for i in range(256):
    for j in range(256):
        enc = E.copy()
        for idx in range(32):
            enc[idx] = ((enc[idx] + i) ^ j) & 0xFF
        if enc.startswith(known_prefix):
            print(f"✅ 找到有效密钥: i={i}(0x{i:02X}), j={j}(0x{j:02X})")
            print(f"    明文: {enc.decode('utf-8', errors='ignore')}\n")

# 第二部分: ((c ^ i) + j) & 0xFF
print("=== 爆破: 先异或后加 ((c ^ i) + j) ===")
for i in range(256):
    for j in range(256):
        enc = E.copy()
        for idx in range(32):
            enc[idx] = ((enc[idx] ^ i) + j) & 0xFF
        if enc.startswith(known_prefix):
            print(f"✅ 找到有效密钥: i={i}(0x{i:02X}), j={j}(0x{j:02X})")
            print(f"    明文: {enc.decode('utf-8', errors='ignore')}\n")

```

用户名是Syclover

验证flag是SYC{F1utt3r_c@n_Us3_ev3rywHer3!}